Pong 게임 프로젝트 - 데이터베이스 설계 문서

프로젝트 요구사항

Major Modules

- Use a framework to build the backend
- Standard user management, authentication, users across tournaments
- Implementing a remote authentication
- Replace basic Pong with server-side Pong and implement an API

Minor Modules

- Use a framework or a toolkit to build the frontend
- Use a database for the backend (SQLite 필수)
- User and game stats dashboards
- Game customization options
- Support on all devices
- Server-Side Rendering (SSR) integration

최종 ERD 설계 (4개 테이블)

1. users 테이블

```
id                  INTEGER PRIMARY KEY AUTOINCREMENT
username            TEXT UNIQUE NOT NULL
email               TEXT UNIQUE NOT NULL
password_hash       TEXT                          -- 로컬 인증용
```

```
oauth_provider      TEXT                          -- 'github', '42', 'google' 등
oauth_id            TEXT                          -- OAuth 사용자 고유 ID
avatar_url          TEXT
created_at          DATETIME DEFAULT CURRENT_TIMESTAMP
updated_at          DATETIME DEFAULT CURRENT_TIMESTAMP

UNIQUE(oauth_provider, oauth_id)                -- OAuth 중복 방지
```

역할:

- 표준 사용자 관리
- 로컬 인증 (username/password)
- 원격 OAuth 인증 (GitHub, 42, Google 등)
- 토너먼트 참가자 정보

2. games 테이블

```
id                  INTEGER PRIMARY KEY AUTOINCREMENT
player1_id          INTEGER NOT NULL REFERENCES users(id)
player2_id          INTEGER NOT NULL REFERENCES users(id)
player1_score       INTEGER DEFAULT 0
player2_score       INTEGER DEFAULT 0
winner_id           INTEGER REFERENCES users(id)
game_settings       TEXT                          -- JSON 형식 커스터마이징 옵션
tournament_id       INTEGER REFERENCES tournaments(id)
status              TEXT DEFAULT 'completed'
created_at          DATETIME DEFAULT CURRENT_TIMESTAMP
ended_at            DATETIME

CREATE INDEX idx_games_player1 ON games(player1_id);
CREATE INDEX idx_games_player2 ON games(player2_id);
CREATE INDEX idx_games_tournament ON games(tournament_id);
```

역할:

- 게임 기록 저장 (최종 결과만)
- 1v1 게임 및 토너먼트 게임 모두 저장
- 게임 커스터마이징 설정 (game_settings JSON)
- 통계 대시보드용 데이터 소스

game_settings 예시:

```
{
  "ball_speed": 1.5,
  "paddle_size": "medium",
  "max_score": 5,
  "court_color": "#000000",
  "paddle_color": "#FFFFFF"
}
```

3. tournaments 테이블

```
id                  INTEGER PRIMARY KEY AUTOINCREMENT
name                TEXT NOT NULL
status              TEXT DEFAULT 'registration'   -- registration, in_progress, completed
max_participants    INTEGER DEFAULT 8
winner_id           INTEGER REFERENCES users(id)
created_at          DATETIME DEFAULT CURRENT_TIMESTAMP
started_at          DATETIME
ended_at            DATETIME
```

역할:

- 토너먼트 정보 관리
- 토너먼트 상태 추적
- 우승자 기록

status 값:

- `registration` : 참가자 모집 중
- `in_progress` : 진행 중
- `completed` : 종료

4. tournament_participants 테이블

```
id                  INTEGER PRIMARY KEY AUTOINCREMENT
tournament_id       INTEGER NOT NULL REFERENCES tournaments(id) ON DELETE CASCADE
user_id             INTEGER NOT NULL REFERENCES users(id)
eliminated          BOOLEAN DEFAULT 0
joined_at           DATETIME DEFAULT CURRENT_TIMESTAMP

UNIQUE(tournament_id, user_id)                  -- 중복 참가 방지
```

```
CREATE INDEX idx_participants_tournament ON tournament_participants(tournament_id);
CREATE INDEX idx_participants_user ON tournament_participants(user_id);
```

역할:

- 토너먼트 참가자 목록
- 탈락 여부 추적
- 사용자별 토너먼트 참가 이력

기술 스택 선택

데이터베이스: SQLite + Prisma ORM

SQLite를 사용하는 이유

- 과제 요구사항 (필수)
- 파일 기반으로 배포 간편
- 설정이 간단함

Prisma를 사용하는 이유

1. 타입 안전성

```
// ❌ Raw SQL - 오타 위험
db.query('SELECT * FROM usres WHERE id = ?', [1]);  // usres 오타!

// ✅ Prisma - 컴파일 타임 에러 체크
const user = await prisma.user.findUnique({
  where: { id: 1 }
});
```

2. 마이그레이션 자동 관리

```
npx prisma migrate dev --name add_tournament
# SQLite 파일 자동 생성 + 버전 관리
```

3. 복잡한 쿼리 간소화

```
// 토너먼트 참가자 목록 + 유저 정보
const tournament = await prisma.tournament.findUnique({
  where: { id: 1 },
  include: {
    participants: {
      include: { user: true }
    }
  }
});
```

SQLite 동시성 문제 해결 방법

1. WAL 모드 활성화 (필수!)

```
import Database from 'better-sqlite3';

const db = new Database('./prisma/dev.db');
db.pragma('journal_mode = WAL');  // 동시 읽기 허용
db.pragma('busy_timeout = 5000'); // 락 대기 시간 5초
```

> **WAL (Write-Ahead Logging) 장점:**
>
> - 읽기 작업이 쓰기 작업을 차단하지 않음
> - 동시성 성능 향상
> - 데이터 무결성 보장

2. 실시간 게임 상태는 메모리에 보관

```
// ❌ 나쁜 예: 게임 상태를 매번 DB 저장
setInterval(() => {
  await prisma.gameState.update({
    ball_x, ball_y, paddle1_y, paddle2_y
  });
}, 16); // 60fps → SQLite 과부하!

// ✅ 좋은 예: 메모리에 보관, 종료 시만 저장
const activeGames = new Map(); // 게임 진행 중 상태
```

```
// 게임 종료 시에만 DB 저장
await prisma.game.create({
  data: {
    player1_id: 1,
    player2_id: 2,
    player1_score: 5,
    player2_score: 3,
    winner_id: 1,
    game_settings: JSON.stringify(settings)
  }
});
```

실시간 게임 아키텍처:

```
WebSocket 연결
     ↓
서버 메모리 (게임 상태)
     ├── ball position
     ├── paddle positions
     ├── current score
     └── game physics
     ↓
게임 종료 시
     ↓
SQLite DB (최종 결과만 저장)
```

Prisma 설정

1. 설치

```
npm install prisma @prisma/client better-sqlite3
```

2. 초기화

```
npx prisma init --datasource-provider sqlite
```

3. schema.prisma

```
datasource db {
  provider = "sqlite"
  url      = "file:./dev.db"
}

generator client {
  provider = "prisma-client-js"
}

model User {
  id            Int      @id @default(autoincrement())
  username      String   @unique
  email         String   @unique
  passwordHash  String?
  oauthProvider String?
  oauthId       String?
  avatarUrl     String?
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt

  gamesAsPlayer1 Game[] @relation("Player1")
  gamesAsPlayer2 Game[] @relation("Player2")
  wonGames       Game[] @relation("Winner")
  tournaments    TournamentParticipant[]
  wonTournaments Tournament[]

  @@unique([oauthProvider, oauthId])
}

model Game {
  id            Int       @id @default(autoincrement())
  player1Id     Int
  player2Id     Int
  player1Score  Int       @default(0)
  player2Score  Int       @default(0)
  winnerId      Int?
  gameSettings  String?   // JSON string
  tournamentId  Int?
  status        String    @default("completed")
  createdAt     DateTime  @default(now())
  endedAt       DateTime?

  player1    User        @relation("Player1", fields: [player1Id], references: [id])
  player2    User        @relation("Player2", fields: [player2Id], references: [id])
  winner     User?       @relation("Winner", fields: [winnerId], references: [id])
  tournament Tournament? @relation(fields: [tournamentId], references: [id])

  @@index([player1Id])
```

```
  @@index([player2Id])
  @@index([tournamentId])
}

model Tournament {
  id              Int       @id @default(autoincrement())
  name            String
  status          String    @default("registration")
  maxParticipants Int       @default(8)
  winnerId        Int?
  createdAt       DateTime  @default(now())
  startedAt       DateTime?
  endedAt         DateTime?

  winner       User?                   @relation(fields: [winnerId], references: [id])
  participants TournamentParticipant[]
  games        Game[]
}

model TournamentParticipant {
  id           Int      @id @default(autoincrement())
  tournamentId Int
  userId       Int
  eliminated   Boolean  @default(false)
  joinedAt     DateTime @default(now())

  tournament Tournament @relation(fields: [tournamentId], references: [id], onDelete: Cascade)
  user       User       @relation(fields: [userId], references: [id])

  @@unique([tournamentId, userId])
  @@index([tournamentId])
  @@index([userId])
}
```

4. 마이그레이션 실행

```
npx prisma migrate dev --name init
```

5. Prisma Studio로 DB 확인

```
npx prisma studio
# http://localhost:5555 에서 GUI로 DB 확인 가능
```

주요 쿼리 예시

사용자 통계 대시보드

```
// 특정 유저의 게임 통계
async function getUserStats(userId: number) {
  const games = await prisma.game.findMany({
    where: {
      OR: [
        { player1Id: userId },
        { player2Id: userId }
      ]
    }
  });

  const totalGames = games.length;
  const wins = games.filter(g => g.winnerId === userId).length;
  const losses = totalGames - wins;
  const winRate = totalGames > 0 ? (wins / totalGames) * 100 : 0;

  const totalScore = games.reduce((sum, game) => {
    return sum + (game.player1Id === userId ? game.player1Score : game.player2Score);
  }, 0);

  const avgScore = totalGames > 0 ? totalScore / totalGames : 0;

  return {
    totalGames,
    wins,
    losses,
    winRate: winRate.toFixed(2),
    avgScore: avgScore.toFixed(2)
  };
}
```

토너먼트 참가자 목록

```
const tournament = await prisma.tournament.findUnique({
  where: { id: tournamentId },
  include: {
    participants: {
      include: {
        user: {
          select: {
            id: true,
```

```
            username: true,
            avatarUrl: true
          }
        }
      },
      where: {
        eliminated: false
      }
    }
  }
});
```

OAuth 로그인 처리

```
async function loginWithOAuth(provider: string, oauthId: string, userData: any) {
  // 기존 유저 찾기
  let user = await prisma.user.findUnique({
    where: {
      oauthProvider_oauthId: {
        oauthProvider: provider,
        oauthId: oauthId
      }
    }
  });

  // 없으면 신규 생성
  if (!user) {
    user = await prisma.user.create({
      data: {
        username: userData.username,
        email: userData.email,
        oauthProvider: provider,
        oauthId: oauthId,
        avatarUrl: userData.avatarUrl,
        passwordHash: null  // OAuth 유저는 비밀번호 없음
      }
    });
  }

  return user;
}
```

API 엔드포인트 설계 (Server-Side Pong)

REST API

```
// 게임 관련
POST   /api/games             // 새 게임 생성
GET    /api/games/:id         // 게임 정보 조회
GET    /api/games             // 게임 목록 (필터/페이징)
GET    /api/users/:id/games   // 특정 유저 게임 기록

// 토너먼트 관련
POST   /api/tournaments       // 토너먼트 생성
GET    /api/tournaments       // 토너먼트 목록
GET    /api/tournaments/:id   // 토너먼트 상세
POST   /api/tournaments/:id/join     // 토너먼트 참가
POST   /api/tournaments/:id/start    // 토너먼트 시작

// 인증 관련
POST   /api/auth/register     // 로컬 회원가입
POST   /api/auth/login        // 로컬 로그인
GET    /api/auth/oauth/:provider     // OAuth 로그인 시작
GET    /api/auth/oauth/:provider/callback  // OAuth 콜백

// 통계 관련
GET    /api/users/:id/stats   // 유저 통계
GET    /api/leaderboard        // 리더보드
```

WebSocket (실시간 게임)

```
// 연결
WS /api/game/:gameId/play

// 메시지 형식
{
  type: 'paddle_move',
  data: { y: 250 }
}

{
  type: 'game_state',
  data: {
    ball: { x: 400, y: 300, vx: 5, vy: 3 },
    paddle1: { y: 250 },
    paddle2: { y: 280 },
    score: { player1: 2, player2: 1 }
  }
}

{
```

```
  type: 'game_end',
  data: {
    winnerId: 1,
    finalScore: { player1: 5, player2: 3 }
  }
}
```

핵심 정리

항목	선택	이유
DB	SQLite	과제 요구사항 (필수)
ORM	Prisma	타입 안전성 + 생산성
동시성	WAL 모드	읽기/쓰기 성능 향상
게임 상태	메모리 저장	DB 부하 최소화
테이블 수	4개	최소 필수 구성

실시간 게임 아키텍처

```
클라이언트 ←→ WebSocket ←→ 서버 메모리 (게임 상태)
                              ↓ (게임 종료 시)
                          SQLite DB (최종 결과)
```

4개 테이블 역할

1. **users**: 인증 + 사용자 관리 (로컬 + OAuth)
2. **games**: 게임 기록 + 통계 소스
3. **tournaments**: 토너먼트 정보
4. **tournament_participants**: 토너먼트 참가 관리

이 설계로 모든 요구사항을 충족하면서도 간결하고 확장 가능한 구조를 만들 수 있습니다! 🎯